

Ejercicio 1 – Construcción de ABB

Se insertan los valores 50, 30, 70, 20, 40, 60 y 80 siguiendo las reglas de un Árbol Binario de Búsqueda, donde los valores menores se ubican hacia la izquierda y los mayores hacia la derecha. El valor 50 se convierte en la raíz del árbol. Luego, 30 se inserta a la izquierda de 50 y 70 a la derecha. Posteriormente, 20 queda ubicado a la izquierda de 30 y 40 a la derecha de 30. Finalmente, 60 se inserta a la izquierda de 70 y 80 a la derecha de 70.

El recorrido In-orden consiste en recorrer primero el subárbol izquierdo, luego la raíz y finalmente el subárbol derecho. Por lo tanto, el recorrido resultante es: 20, 30, 40, 50, 60, 70 y 80.

La altura del árbol medida en aristas es 2, ya que el camino más largo desde la raíz hasta una hoja contiene dos aristas.

Ejercicio 2 – Traza de Búsqueda

La búsqueda del valor 55 comienza en la raíz del árbol. Primero se compara con 50 y, como 55 es mayor, la búsqueda continúa hacia la derecha. Luego se compara con 70 y, como 55 es menor, la búsqueda continúa hacia la izquierda. Finalmente se compara con 60 y, como 55 es menor, debería continuar nuevamente hacia la izquierda. Sin embargo, la referencia izquierda de 60 es null, lo que indica que el valor no existe dentro del árbol.

La secuencia de nodos visitados durante la búsqueda es: 50 → 70 → 60.

La cantidad total de comparaciones de clave realizadas es 3.

Ejercicio 3 – Eliminación en ABB

El nodo 30 posee dos hijos: 20 y 40. Por esta razón, corresponde al caso de eliminación de un nodo con dos hijos. Aplicando el criterio del sucesor In-orden, se busca el menor elemento del subárbol derecho del nodo 30. En este caso, el sucesor In-orden es el valor 40.

El valor 40 reemplaza al nodo 30 y luego el nodo original 40 se elimina de su ubicación anterior. Como resultado, el árbol mantiene correctamente la propiedad de ABB.

Ejercicio 4 – Factores de Balance AVL

Luego de insertar los valores 30, 20, 40, 10 y 25, el árbol queda balanceado. El nodo 30 funciona como raíz, teniendo a 20 como hijo izquierdo y a 40 como hijo derecho. A su vez, el nodo 20 posee como hijos a 10 y 25.

Luego de insertar los valores 30, 20, 40, 10 y 25, el árbol queda balanceado. El nodo 30 funciona como raíz, teniendo a 20 como hijo izquierdo y a 40 como hijo derecho. A su vez, el nodo 20 posee como hijos a 10 y 25.

El Factor de Balance se calcula utilizando la fórmula $bf = h(\text{izquierda}) - h(\text{derecha})$, considerando que la altura de un nodo null es -1.

Los nodos 10, 25 y 40 no poseen hijos, por lo que su factor de balance es 0. El nodo 20 tiene la misma altura en ambos subárboles, por lo tanto, su factor de balance también es 0. Finalmente, el nodo 30 tiene una altura izquierda de 1 y una altura derecha de 0, obteniendo un factor de balance igual a 1. Todos los nodos poseen factores comprendidos entre -1 y 1, por lo que el árbol se considera balanceado.

Ejercicio 5 – Rotación AVL: Caso LL

El árbol AVL contiene inicialmente los valores 30 y 20. Luego se inserta el valor 10, quedando una estructura inclinada hacia la izquierda. El nodo 30 se desbalancea debido a que el subárbol izquierdo posee mayor altura que el derecho.

El Factor de Balance del nodo 30 es 2, lo que indica un desbalance del tipo LL (Left-Left), ya que la inserción ocurrió en la izquierda del hijo izquierdo. Para corregir este problema se aplica una rotación simple hacia la derecha sobre el nodo 30. Luego de la rotación, el árbol queda balanceado con el nodo 20 como nueva raíz, 10 como hijo izquierdo y 30 como hijo derecho.

Ejercicio 6 – Rotación AVL: Caso LR

El árbol AVL contiene inicialmente los valores 30 y 20. Luego se inserta el valor 25. El nuevo nodo queda ubicado en la derecha del hijo izquierdo de la raíz, provocando un desbalance en el nodo 30.

El Factor de Balance del nodo 30 es 2 y corresponde a un caso LR (Left-Right). La solución requiere una rotación doble. Primero se aplica una rotación izquierda sobre el nodo 20 y luego una rotación derecha sobre el nodo 30. Después de ambas rotaciones, el árbol queda balanceado con 25 como raíz, 20 como hijo izquierdo y 30 como hijo derecho.

Ejercicio 7 – TDA Árbol B: Reglas

En un Árbol B de orden 3, cada nodo puede tener como máximo 3 hijos. Como la cantidad máxima de claves siempre es igual al orden menos uno, cada nodo puede almacenar hasta 2 claves. La cantidad mínima de claves para un nodo que no sea raíz es 1. Esta restricción permite mantener el árbol balanceado y evita la existencia de nodos vacíos.

Ejercicio 8 – Inserción en Árbol B

Al insertar los valores 10 y 20 en un Árbol B de orden 3, ambos pueden almacenarse dentro de la raíz sin inconvenientes. Sin embargo, al insertar el valor 30, el nodo supera la cantidad máxima permitida de claves, generando una condición de overflow.

El nodo queda temporalmente con las claves 10, 20 y 30. Para resolver el overflow, la clave central, que es 20, se promociona y pasa a convertirse en la nueva raíz del árbol. Luego, las claves restantes se dividen en dos nodos hoja: uno contiene el valor 10 y el otro contiene el valor 30. Como resultado, la estructura final queda balanceada con 20 en la raíz y dos hojas separadas que contienen 10 y 30 respectivamente.